Operations

Run it day to day, upgrade it, fix problems.

For: Administrators and engineers

One chapter of the handbook. The full handbook, with every chapter, is handbook.pdf in the same folder.

Day-to-day running, upgrades, backups and troubleshooting.

Everyday commands

```
docker compose ps                                  # what's running and healthy
docker compose logs -f --tail=100 policy-api       # follow one service
docker compose restart policy-api                  # after editing .env
docker compose stop && docker compose up -d        # full restart
scripts/check_containment.sh                       # after any change
```

Optional services need their profile: `docker compose --profile escalate up -d escalate`.

Health

Check	Healthy looks like
`curl -s http://127.0.0.1:8100/health`	`{"ok":true,"demo_mode":false}` (if `demo_mode` is true, real models are **not** in use)
`docker compose ps`	`db`, `policy-api` and `granite-gateway` show `healthy`
`curl -s $GRANITE_URL/v1/models`	Lists `granite-4.2-30b`. Granite runs on its own server, so check it there (and with whoever runs it), not with `nvidia-smi` here
Recent runs	`SELECT status, count(*) FROM runs WHERE created_at > now() - interval '1 day' GROUP BY 1;` shows few `failed`
Job queue	`curl -s -H "Authorization: Bearer $POLICY_API_KEY" http://policy-api:8000/queue` from inside the network: `oldest_wait_s` near 0 and `dead` not rising

The job queue

Every check is a job in the `jobs` table. `policy-api` runs a worker that takes jobs one at a time per free slot (`WORKER_CONCURRENCY`, default 2) and renews a lease on each while it works (`JOB_LEASE_SECONDS`, default 60).

- **Restarts and crashes don't lose checks.** If `policy-api` restarts, it takes back its own jobs at once. If it dies or the server reboots, the lease runs out and the job is picked up again. The check starts over from the beginning; people waiting in the chat see "Starting again after an interruption...".

- **Temporary problems are retried** (a model server restarting, the database briefly away), up to `JOB_MAX_ATTEMPTS` (default 3) with growing waits. After that the job is `dead` and the person sees "failed several times in a row".
- **Problems with the question itself are not retried** (no such policy, nothing changed).
- **More capacity:** add workers with `docker compose run -d --name worker-2 policy-api python -m app.worker`, or raise `WORKER_CONCURRENCY`. Workers never take the same job twice. Set `WORKER_ENABLED=0` on `policy-api` if you want only separate workers to run checks.

Dead jobs keep their last error:

```
SELECT id, kind, payload, attempts, last_error, finished_at FROM jobs WHERE status = 'dead' ORDER
BY id DESC LIMIT 20;
-- Retry one after fixing the cause:
UPDATE jobs SET status = 'pending', attempts = 0, run_at = now() WHERE id = <id>;
UPDATE runs SET status = 'queued', error = NULL, finished_at = NULL WHERE id = '<payload
run_id>';
-- Housekeeping (monthly):
DELETE FROM jobs WHERE status = 'done' AND finished_at < now() - interval '30 days';
```

How long a run takes

Roughly: number of changed sections × about 20–40 related clauses × time per verdict ÷ `JUDGE_CONCURRENCY`. For example, 10 changed sections with 30 clauses each is 300 verdicts; at 16 at a time and about 3 s each, that's about one minute. Raise `JUDGE_CONCURRENCY` if the Granite server isn't busy; lower it if verdicts time out.

Backups

Back up the database daily. The models and the policy source files are recoverable from their original sources; runs and verdicts are not.

```
docker compose exec -T db pg_dump -U policy -Fc policy > backups/policy-$(date +%F).dump
# Restore into an empty database:
docker compose exec -T db pg_restore -U policy -d policy --clean --if-exists < backups/policy-
YYYY-MM-DD.dump
```

Test a restore every quarter on a spare machine. Also back up the `openwebui` volume (users and the upload filter's settings), and keep `.env` in your secrets vault.

Upgrades

Change one thing at a time, and run the evaluation after anything that touches the models.

1. Read the release notes for the component.
2. Bring the new image or weights in as in setup.md; record and check hashes.
3. Change the tag in `.env` (or the folder in `compose.yaml`).
4. `docker compose up -d <service>`
5. `scripts/check_containment.sh`

6. Smoke test (testing.md), and evaluate if a model changed.
7. Roll back by restoring the old tag and running `docker compose up -d <service>`.

Granite upgrades happen on the Granite server, not here. After any change there (vLLM version, flags, model files), run `curl -s $GRANITE_URL/v1/models`, then the evaluation. To change the key or address, edit `GRANITE_API_KEY` / `GRANITE_URL` in `.env` and run `docker compose up -d granite-gateway`.

Open WebUI upgrades can change how files reach the upload filter. Afterwards, attach a file and ask a question. If the assistant says no file was attached, check how the new release passes `__files__` and adjust `openwebui/upload_filter.py`.

Hermes upgrades: Hermes' setting names can change between releases. Read the release notes, then compare `hermes/config.yaml` with the new release's defaults before changing `HERMES_TAG`. After `docker compose up -d hermes`, repeat the checks in setup.md: only the policy tools are listed, and a question gets a cited answer.

policy-api upgrades: rebuild outside the environment with a new `APP_VERSION`, carry the image in, and set `APP_VERSION` in `.env`. Checks in progress during the restart are picked up again automatically. Database changes in `policy_api/app/migrations/` are applied on start, once each.

Changing the embedding model

The `clauses.embedding` column has a fixed size (768). To switch to a model with a different size:

```
docker compose exec db psql -U policy -c "DROP INDEX clauses_embedding_idx;
  ALTER TABLE clauses ALTER COLUMN embedding TYPE vector(<new size>) USING NULL;
  CREATE INDEX clauses_embedding_idx ON clauses USING hnsw (embedding vector_cosine_ops);"
# Put the new model in models/embedding, restart embed, then re-ingest every policy:
docker compose up -d embed && docker compose run --rm policy-api python -m app.ingest /policies
```

Update `vector(768)` in `db/init.sql` too, so new installs match. Re-run the evaluation.

Moving a model to different GPUs

Run `python3 scripts/setup_gpus.py` again and give the new GPU numbers. Listing several GPUs for `escalate` (or another vLLM model run by this stack) spreads that model across them (tensor parallelism); the script sets `ESCALATE_TP` and the other size settings to match. Then `docker compose up -d escalate` (or whichever service moved). Granite's GPUs are managed on its own server.

Troubleshooting

Symptom	Likely cause	Fix
policy-api exits with `Missing settings`	`.env` incomplete	Fill in the named variables
policy-api exits with `at least 32 characters`	Short key	Regenerate with `openssl rand -hex 32`

Symptom	Likely cause	Fix
`400 Invalid host header`	The Host name isn't in `ALLOWED_HOSTS`	Add the name the proxy sends (`policy-api`) or your hostname
The assistant says "The policy service did not respond"	policy-api down, or `POLICY_API_KEY` differs between `policy-api` and `hermes`	`docker compose ps` ; `docker compose logs hermes policy-api` ; restart both after changing the key
Open WebUI shows a model error, or no `policy-assistant` model	`hermes` is down, or `HERMES_API_KEY` differs between Hermes and Open WebUI (they read the same `.env` value)	`docker compose logs hermes` ; `docker compose up -d hermes open-webui`
`hermes` exits at start	`HERMES_API_KEY` missing or under 16 characters, or Granite's `--max-model-len` is under 65536	Fix `.env` or the Granite server; see setup.md
Chat says "checking model is not responding"	The Granite server is down or overloaded, or `granite-gateway` cannot reach it	`docker compose logs granite-gateway` ; run `curl -s $GRANITE_URL/v1/models` ; lower `JUDGE_CONCURRENCY`
Many "could not be read" verdicts	Structured output not working with this vLLM version	Check the Granite server's vLLM version supports `response_format` JSON schema; check its logs
The assistant answers from memory, without calling a tool	Granite's server lacks `--enable-auto-tool-choice` and a tool-call parser, or the plugin did not load	Check the flags (setup.md); `docker compose logs hermes` should show the `secretary` plugin loaded
"No numbered sections were found" for a Word file	Word automatic numbering	Save with typed numbers, or as PDF (how-to.md)
Download link gives 403	Link expired (72 h) or copied incompletely	Ask the assistant again
Download link gives a connection error	`PUBLIC_BASE_URL` isn't reachable from users' browsers	Set it to the proxy's public address and restart policy-api
A run stays "running"	It's large, or Granite is slow	Check `progress` on the run; the plugin stops waiting after `SECRETARY_MAX_WAIT_MINUTES` (10) but the run continues; ask the assistant for the result again
Ingest skips files	No ID, no numbered sections, or unreadable type	The skip reason is printed for each file
`check_containment.sh` fails	A service can reach the internet	security.md